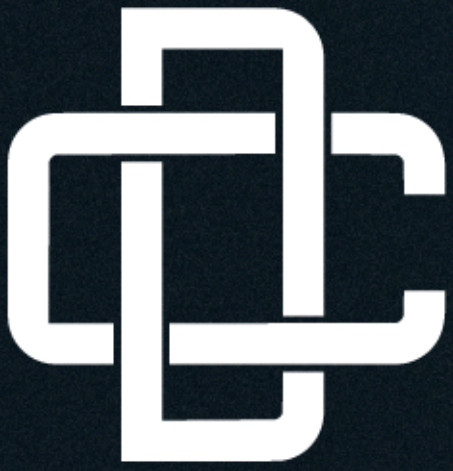

CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Beezie

July 2026

Prepared by
0xluk3
zeroXchad

Introduction

A time-boxed security review of the **Beezie** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About **Beezie**

Beezie is a Web3 collectibles platform built around on-chain "claw machine" games, where users pay to play and win tokenized real-world collectibles that are minted as NFTs and drawn from a weighted prize pool. It runs on Solana and EVM chains (Flow and Base) with a provably-fair commit-reveal randomizer deciding each pull, plus a secondary marketplace, raffles, and points system for buying, trading, and giving away those collectibles.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

***review commit hash* - c4cec339163312039680cf08ac8948d925711ad4**

***fixes review commit hash* - 34d2bec6b9501c82f00437ce8a35e4a338b2777f**

Scope

The following files were in scope of the audit:

- `packages/provable-pick/src/provableWeightedRandomizer.ts`
- `apps/lucky-api-solana/src/services/solana/randomness.ts`

The following number of issues were found, categorized by their severity:

- Critical & High: 0 issues
- Medium: 1 issues
- Low: 2 issues
- Informational: 3 issues

Findings Summary

ID	Title	Severity	Status
[M-01]	`verifyPick` authenticates only the server seed, so selection integrity is not verifiable	Medium	Fixed
[L-01]	Config validation is not enforced at runtime	Low	Fixed
[L-02]	Two different libraries with Keccak implementation in use	Low	Fixed
[I-01]	`hex32ToBytes` validates length but not character set, allowing silent truncation	Informational	Fixed
[I-02]	Possible cross-chain hash re-usage	Informational	Fixed
[I-03]	Incorrect variable in `hex32ToBytes` error message	Informational	Fixed

Detailed Findings

[M-01] `verifyPick` authenticates only the server seed, so selection integrity is not verifiable

Severity

Impact: High

Likelihood: Low

Description

The claw machine is presented as a provably-fair draw. A provably-fair draw requires that any observer be able to reconstruct the outcome from inputs that were committed before the entropy became known, so that neither party can influence the result afterwards. Two things must be committed for this to hold. The first is the randomness, which here is the server seed. The second is the mapping from that randomness

to a concrete prize, which is the eligible item set, its order, and the odds. The implementation commits only the first.

When a user starts a play, the server generates a random seed and publishes only its hash on-chain as the commitment. In `randomness.ts`, `createServerSeedCommitment` produces the seed and that hash.

```
const serverSeed = new Uint8Array(randomBytes(32));
const seedCommitment = keccak_256(serverSeed);
```

The raw server seed is stored in the database in plaintext and disclosed only later, while `seedCommitment` is what goes on-chain. This is a sound commitment, because the hash proves the seed was fixed before the entropy block existed.

Later, to pick the reward, the cranker calls the picker with the seed, the now-known block hash, and the item pool and odds loaded from the operator's database. In `provableWeightedRandomizer.ts`, `pickSequence` runs one weighted draw per pull. Each step derives a uniform `bucket` from the seed and the block hash, then selects an item by its position in the pool.

```
const bucket = uniformFromHash(
  input.serverSeed,
  input.playBlockHash,
  input.playId,
  step,
);
const idx = weightedIndex(weights, bucket);
chosen.push(pool[idx]);
resultIndices.push(idx);
pool.splice(idx, 1);
```

The chosen items become concrete NFTs and are transferred to the user by the on-chain reveal. In `clawCranker.ts`, the reveal instruction receives the seed and the already-chosen mints and nothing else from the draw.

```
buildRevealInstruction({
  // ...
  serverSeed,
  reveals: batch.map((mint) => ({ asset: new PublicKey(mint) })),
});
```

The problem is that the pool and the odds decide which item each bucket maps to, yet neither is committed. Both are read from operator-controlled database rows at reveal time. The awarded item also depends on the order of the pool, because `idx` is a position rather than an identity. `verifyPick` inherits this gap. It confirms the seed matches its commitment and then replays the draw against whatever pool and odds it is given, without checking that either was fixed in advance.

`verifyPick` is defined in `packages/provable-pick/src/provableWeightedRandomizer.ts`.

```
export function verifyPick(
  input: ProvablePickInput,
  seedCommitment: `0x${string}`,
  recordedResultIndices: number[],
): boolean {
  if (keccak256(input.serverSeed) !== seedCommitment) return false;
  const replay = pickSequence(input);
  if (replay.resultIndices.length !== recordedResultIndices.length)
    return false;
  return replay.resultIndices.every((v, i) => v ===
recordedResultIndices[i]);
}
```

By the time the draw runs, the operator already knows both entropy inputs, namely the plaintext seed and the settled block hash. It can reorder the pool or adjust the odds until the fixed bucket lands on the cheapest item, award that item on-chain, and still pass `verifyPick`. The recorded positions in `resultIndices` never reach the chain, so the on-chain program cannot constrain which assets are transferred.

The draw therefore fails requirement P1, which holds that the operator cannot predetermine or steer the prize. An operator can decide which prize a paying player receives while every fairness check continues to pass, and the player has no way to detect it.

Recommendations

Commit the selection inputs alongside the seed. At play creation, publish a hash or Merkle root of the sorted item set and the odds into the on-chain `PlayerState` next to `seed_commitment`, and require `verifyPick` to check those inputs against that commitment before replaying. Sorting the item set by stock id makes its order fixed rather than a free parameter.

Client

Fixed via removal.

[L-01] Config validation is not enforced at runtime

Description

The library exposes `pickSequence` as its public API. However it never performs `assertValidPickConfig` internally. The library therefore relies on every caller remembering to validate. Since this module is intended to provide deterministic provable randomness, allowing invalid configuration objects to reach the picker weakens the trust assumptions. A security-sensitive library should enforce its own invariants.

```
export function pickSequence(input: ProvablePickInput): WeightedPickResult
{
  if (input.items.length < input.times) {
    throw new Error(
      `insufficient eligible stock: have ${input.items.length}, need
${input.times}`,
    );
  }
  return isTierWeightedConfig(input.config)
    ? pickSequenceTiered(input, input.config)
    : pickSequenceEvBand(input, input.config);
}
```

Recommendations

Call the `assertValidPickConfig(input.config)` function at the start of `pickSequence`.

Client

Fixed.

[L-02] Two different libraries with Keccak implementation in use

Description

The `randomness.ts` uses `@noble/hashes`, `verifyPick` uses the `viem` library. A mismatch in implementations may silently break the implementation. Even if now both implementations are equal, a future upgrade might not persist this invariant.

```
// provableWeightedRandomizer.ts
import { encodePacked, keccak256 } from "viem";
```

```
// randomness.ts
import { keccak_256 } from "@noble/hashes/sha3";
import { randomBytes } from "crypto";
```

Recommendations

It is recommended to use a single library for cryptography-related functionalities.

Client

Fixed.

[I-01] `hex32ToBytes` validates length but not character set, allowing silent truncation

Description

`hex32ToBytes` in `apps/lucky-api-solana/src/services/solana/randomness.ts` strips the `0x` prefix, checks the string is 64 characters, then converts it with `Buffer.from(normalized, "hex")` without validating that the characters are valid hex. Node's hex decoder stops at the first invalid character and returns a buffer shorter than 32 bytes with no error, so malformed input is silently truncated rather than rejected.

```
export function hex32ToBytes(hex: string): Uint8Array {
  const normalized = hex.startsWith("0x") ? hex.slice(2) : hex;
  if (normalized.length !== 64) {
    throw new Error(`expected 32-byte hex string, got ${hex.length}
chars`);
  }
  return new Uint8Array(Buffer.from(normalized, "hex"));
}
```

No current caller supplies malformed input. The server seed is generated by a CSPRNG, and the one user-supplied value that could reach this function, the discount code hash, is already validated against a strict character pattern upstream. The worst case today is a failed transaction with no benefit to an attacker. If a future caller passes unvalidated hex into this function, it would operate on truncated data instead of stopping with an error.

Recommendations

Validate the character set with a pattern such as `/^[0-9a-fA-F]{64}$/` before decoding, and assert that the decoded result is exactly 32 bytes.

Client

Fixed.

[I-02] Possible cross-chain hash re-usage

Description

The solution appears to be using namespace separation between EV mode and tier mode. However, assuming these values and overall algorithm are going to be used within the smart contract, the current namespace separation does not necessarily protect from cross-chain or cross-contract hash re-usage, or possible signature replay. While `playBlockHash` possibly protects from re-usage within the same chain, it does not protect from possible re-usage on a different chain. However, this risk heavily depends on the solution design and assumptions, e.g. the number of deployments on different chains.

```
export function uniformFromHash(
  serverSeed: `0x${string}`,
  playBlockHash: `0x${string}`,
  playId: number,
  step: number,
): bigint {
  const h = keccak256(
    encodePacked(
      ["bytes32", "bytes32", "uint256", "uint256"],
      [serverSeed, playBlockHash, BigInt(playId), BigInt(step)],
    ),
  );
  return BigInt(h) % UNIFORM_BUCKETS;
}
```

```
export function uniformFromHashPhase(
  serverSeed: `0x${string}`,
  playBlockHash: `0x${string}`,
  playId: number,
  step: number,
  phase: number,
): bigint {
  const h = keccak256(
    encodePacked(
      ["bytes32", "bytes32", "uint256", "uint256", "uint256"],
      [serverSeed, playBlockHash, BigInt(playId), BigInt(step),
BigInt(phase)],
    ),
  );
  return BigInt(h) % UNIFORM_BUCKETS;
}
```

Recommendations

It is recommended to implement principles from EIP-712 domain separation to prevent cross-chain re-usage of hashes.

Client

Fixed.

[I-03] Incorrect variable in `hex32ToBytes` error message

Description

Within the `hex32ToBytes` function the condition checks the `normalized.length` variable, however the error message reports `hex.length`. When the input carries a `0x` prefix, `hex.length` and `normalized.length` differ by two characters, so the length reported in the error is misleading and may complicate debugging.

```
export function hex32ToBytes(hex: string): Uint8Array {
  const normalized = hex.startsWith("0x") ? hex.slice(2) : hex;
  if (normalized.length !== 64) {
    throw new Error(`expected 32-byte hex string, got ${hex.length}
chars`);
  }
  return new Uint8Array(Buffer.from(normalized, "hex"));
```

Recommendations

Use `normalized.length` in the error message so the reported length matches the value that was actually validated.

Client

Fixed.